

Spark vs Ray — Benchmark Results

Vishnu Vardhan
DA24B038

October 5, 2026

1 Benchmark Setup

Two distributed pipelines — **Apache Spark 3.5.1** and **Ray 2.59.0 (Ray Data)** — were executed on identical hardware (Ubuntu 24.04 VM, 2 vCPU, 3.8 GB RAM) over the same NYC Yellow Taxi January 2024 dataset (296,462 rows + 265 zone lookup rows). Both frameworks were configured with **2 worker CPUs**:

- **Spark:** 1 master + 2 workers × 512 MB, driver 500 MB, 1 core per worker.
- **Ray:** Head node with 2 CPUs, 476 MB object store, 2.76 GiB memory pool.

The pipeline in both frameworks performs: *ingest* → *cleanse* → *heavy join* → *Python UDF (speed per hour)* → *Parquet export*.

2 Execution Time Results

Stage	Spark (s)	Ray (s)	Faster
Init (session/cluster start)	33.99	4.11	Ray
Ingest	63.84	3.22	Ray
Cleanse	17.19	20.44	Spark
Heavy Join	51.04	22.57	Ray
UDF + groupBy	26.91	62.14	Spark
Export (write Parquet)	71.17	0.22	Ray
Total wall-clock	291.45	117.33	Ray (2.48×)

Table 1: Per-stage and total execution time. Ray is 2.48× faster end-to-end.

Interpretation

Ray is dramatically faster on *startup* (4.11 s vs 33.99 s) because it has no JVM boot cost and no cluster master/worker JVM hierarchy. Ray also dominates on *ingest* and *export*, where its object-store backed zero-copy I/O beats Spark’s JVM-serialized shuffle paths. Spark wins narrowly on the *cleanse* stage (17.19 s vs 20.44 s) due to vectorized DataFrame operations compiled into JVM bytecode.

The overall winner is clearly Ray: **117.33 s vs 291.45 s**, a 2.48× speedup.

3 Python UDF Overhead — Deep Dive

The assignment’s central question: *is Ray’s Python-native UDF path faster than Spark’s JVM-bridged path?*

Stage	Spark (s)	Ray (s)
UDF + groupBy	26.91	62.14
Ratio	1×	2.31× slower

Table 2: UDF stage timing. Spark is 2.31× faster on this dataset.

4 Pipeline Parity Verification

Both pipelines produced **177 result rows** (aggregated by `hour_of_day` × `Borough`). An outer merge across all rows shows:

Metric	Value
Spark rows	177
Ray rows	177
Rows compared (outer join)	201
Max $ \Delta avg_speed_mph $	5.79×10^{-5}
Mean $ \Delta avg_speed_mph $	4.85×10^{-7}

Table 3: Parity verification between Spark and Ray outputs.

The maximum difference across all matched rows is **0.000058 mph**, which is low. **Pipeline parity is verified.**

5 Per-Hour Comparison (Sample)

Hour	Spark speed	Ray speed	Match?
0	43.2808	43.2808	
6	82.2574	82.2574	
11	292.0977	292.0977	
15	46.6280	46.6280	
18	279.8766	279.8766	
23	99.8131	99.8131	

Table 4: Sample of per-hour average speeds. All 24 hours match to 4 decimal places.

Criterion	Spark	Ray
Total wall-clock	291.45 s	117.33 s
Cluster startup	33.99 s	4.11 s
Python UDF stage	26.91 s	62.14 s
Architecture footprint	4 JVMs	1 head node
Pipeline parity	Identical (max diff 5.8×10^{-5})	

Table 5

6 AI Usage

- Whenever i got an error or don't have an idea of what to do, i asked AI what to do.
- I did write commands even though sometimes i don't know what it does exactly.